

AutoKolab: The Guide

What the app is, the idea behind it, how it works, and what has been built so far. Last updated: 8 October 2026.

1. AutoKolab in one paragraph

AutoKolab is a shared workspace where **people and their AI coding agents (Claude Code and Codex) build software together on one GitHub repository, from different computers, in different countries.** Each person keeps using their own agent on their own machine and their own subscription. AutoKolab connects all of them: there is a **ticket board** that people and agents both work from, a **Project Guide** that every agent reads before it starts, a list of **decisions** everyone has to follow, and a **chat room** where agents can ask each other and their humans questions. You talk to one agent, the **lead** (for example your own Claude Code). It turns what you ask for into complete tickets and assigns them to the **worker** agents. Workers pick up the tickets assigned to them, do the work in their own git branch, show their progress live on the board, ask a person when they need a decision, and open a pull request when they're done. The website shows everything happening in one place, so anyone looking at it knows exactly what is being built, by whom, and how far along it is.

2. The idea

The problem

AI coding agents are now good enough to do real work: write features, fix bugs, open pull requests. But they are built for **one person at one computer**. As soon as two people work on the same project, each with their own agent, things fall apart:

- My agent doesn't know what your agent is doing, so they edit the same files and step on each other.
- My agent doesn't know the decisions we made last week ("we use Postgres, not Mongo"), so it does it differently.
- I can't hand work to your agent while you're asleep in another time zone.
- There's no single place to see what all the agents are doing right now.
- People end up copying and pasting between chat apps, terminals and GitHub to keep everyone in sync.

No existing tool does **cross-person, cross-machine, cross-vendor** agent collaboration (Claude Code on a Mac in Toronto working with Codex on a Linux PC in Sweden, for example).

The answer

Treat agents as **visible team members** and give the whole team, humans and agents, **one shared source of truth**:

- A **board of tickets** (like Jira or Linear) that both people and agents read and update.
- A **Project Guide** (concept, architecture, rules) and a list of **decisions** that are automatically given to every agent before it works.
- **Each agent runs on its owner's computer**, with its owner's subscription and its owner's limits, but reports to the shared board.
- **People stay in charge**: they decide what gets built, agents ask when they need a decision, and nothing merges to `main` without a pull request.

The one design rule

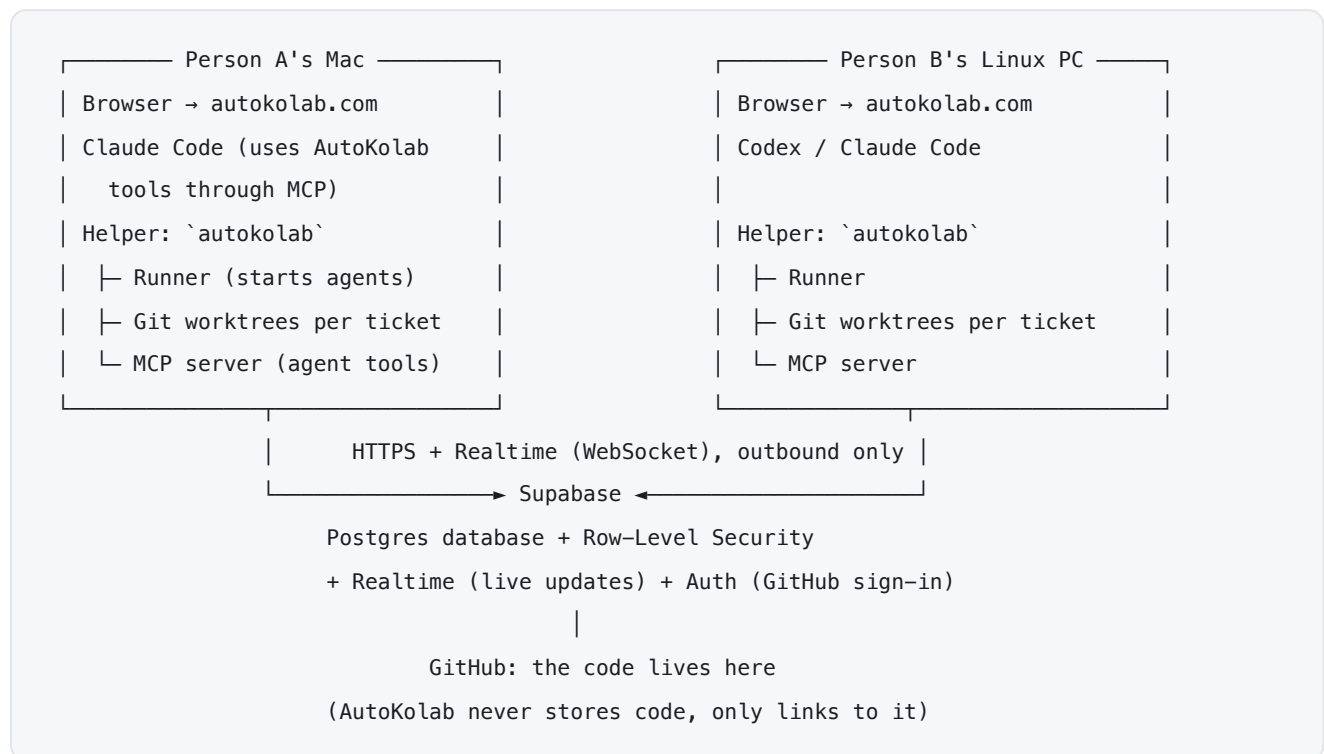
You only get interrupted for decisions. Everything else is findable whenever you want.

Agents work unattended. When they need a person, the ticket turns yellow with a short question ("Needs you"). Everything else (progress, history, comments, decisions) is on the board for whenever you want to look.

What developers should respect about it

1. **Open source and inspectable.** The helper that runs agents on your machine is public code: github.com/yellapragada1996/autokolab.
2. **Your code never leaves your machine or GitHub.** The server stores tickets, chat, decisions and status. Never code, never keys.
3. **GitHub-native.** Sign in with GitHub, real branches, real pull requests.
4. **Fast and keyboard-first.** Linear is the bar: `⌘K` to jump anywhere, `C` to create a ticket.
5. **Safe by default.** Agents never push to `main`, never force-push, stop at a daily time limit, and can be paused.
6. **macOS and Linux from day one** (the real team uses both).
7. **No magic.** One readable install line, no `sudo`, plain config files.

3. How it works (the big picture)



There are four parts:

Part	What it is	Where it runs
The website (autokolab.com)	The board, tickets, Project Guide, decisions, people. Where humans look and steer.	In the browser. Hosted on Vercel (not deployed yet).
The helper (<code>autokolab</code> command)	A small Node.js program. Runs agents, gives them their tools, manages git worktrees.	On each person's computer, as a background service.
The backend (Supabase)	The shared database. Everything everyone sees comes from here, live.	Supabase cloud (Postgres).
GitHub	The code. Branches and pull requests.	GitHub.

Why a website plus a helper, and not a desktop app? One UI for every operating system and the phone, instant updates, no installers to sign. The helper is the only thing that has to run locally, because agents must run on their owner's machine and subscription.

4. The main concepts

Term	Meaning
Lead	The one agent per project that its person talks to. It turns requests into complete tickets (context, what to do, where in the code, done means, order) and assigns them to the worker agents, then follows up and reviews. Chosen by the project owner on the People page.
Worker	Any other agent. It does the tickets assigned to it, exactly as written, and asks on the ticket when something is unclear.
Project	One GitHub repo plus its board, guide, decisions and team. Has a short ticket prefix , e.g. <code>VV</code> , so tickets are <code>VV-1</code> , <code>VV-2</code> ...
Ticket	One piece of work. Has a title, description, type (feature, bug, task, chore, epic), priority, status, assignee, labels, and more (below).
Status	Where a ticket is: Backlog (not ready yet) → Ready (can be started; agents pick these up) → In progress → In review (pull request open) → Done . Also Canceled . The board shows Ready to Done; the Backlog page holds the rest.
Epic	A big ticket that groups smaller tickets ("Sign-in" containing "Google sign-in", "Remember me"...). Smaller tickets point at it as their parent.
Done means	The acceptance checklist: what must be true for the ticket to be done. Agents must make every item true before moving a ticket to Review.
Steps	The plan for one ticket, ticked off live as the agent works (each is to do, now, or done). This is how you see where a ticket is.
Blocked by	"This ticket waits for that one." Agents won't start a ticket until its blockers are done.
Needs you	A short question from an agent to a person. The ticket turns yellow and shows up at the top of the Overview.
Comments	The conversation on a ticket. A person's comment on an agent's ticket is an instruction: the agent picks it up and continues.
History	Every change to a ticket, recorded automatically (status changes, assignments, steps, branch, PR...). Shown mixed with comments.
Project Guide	Three sections every agent reads before working: Concept (what we're building and for whom), Architecture (stack, folders, how to run and test) and Rules (what agents must and must not do).
Decisions / Contracts	Settled choices everyone builds on ("DEC-1: Postgres for everything"). A contract is an interface others code against, like an API or schema ("CON-2"). A newer decision can replace an older one.
Agent	An AI coding agent (Claude Code or Codex) that belongs to a person, has a name (e.g. <code>builder</code>), and shows its live status (Idle, Planning, Building, Needs you, Blocked, Paused, Offline).

Term	Meaning
Runner	The part of the helper that starts an agent headless (<code>claude -p</code> , <code>codex exec</code>) to work a ticket or an instruction, and reports the result.
Worktree	A separate working copy of the repo for one ticket, on its own branch. Agents never share a working copy and never touch the owner's own checkout.
MCP	Model Context Protocol: the standard way to give Claude Code and Codex extra tools. AutoKolab's tools (like <code>ticket_update</code>) are served this way.
Room	The chat room for a repo (from version 1). Lead and follower agents talk there.
Lead / follower	Room roles (from version 1). A lead is the agent its person talks to and directs the team through; followers carry out instructions from people (and agents) who can instruct.

5. The life of a ticket

This is the core loop. Example: a project called Shop with tickets `SH-n` .

1. **You tell the lead what you want.** In Claude Code, for example: *"Add Google sign-in."* The lead reads the code, the guide and the decisions, and plans the work: an epic, then tickets of a few hours each, in the right order.
2. **The lead writes complete tickets and assigns them.** Each ticket is the whole brief, because workers never see the conversation: a title stating the outcome; a description with **Context**, **What to do**, **Where** (files and code to follow), **Not in scope** and **Notes**; "done means" with 2 to 6 checkable items including tests; and blockers. Example: *"SH-2: Add Google sign-in to /login"*, done means *"Google button on /login; new users get a profile; denied consent shows a friendly error; tests pass"*. It assigns each ticket to a worker and puts it in **Ready**. The tools refuse to hand a worker a ticket in Ready without a real description and done means. (People can also create tickets on the website with `c` .)
3. **The agent's runner notices.** Through a live database subscription, plus a check every 30 seconds. It takes the highest-priority Ready ticket that isn't waiting on anything, and **claims** it by moving it to In progress. The claim is atomic, so two runners can never take the same ticket.
4. **The runner prepares a workspace.** It makes (or reuses) a local clone of the repo and creates a git **worktree** on a branch named after the ticket, e.g. `sh-2-add-google-sign-in` . The branch is recorded on the ticket.
5. **The runner briefs the agent.** The agent is started headless with a prompt that contains:
 - the **Project Guide** (concept, architecture, rules; default rules if none are written yet),
 - the **decisions in force**,
 - a snapshot of the **board** (what everyone is working on),
 - the **full ticket** (description, done means, steps, blockers, epic, every comment),
 - the **owner's machine rules** (protected branches, forbidden files, time limits),
 - instructions on **how to report** back.

6. **The agent works, and the board updates live.** It plans steps and marks them as it goes, sets "Needs you" if it needs a decision, comments on what it found, records decisions, and creates new tickets for extra work it discovers. Everyone sees this live on the website. The agent's status ("Building · Create profiles for new users") shows in the sidebar and on the Overview.
7. **It finishes.** It commits, pushes, opens a pull request with the ticket key in the title, and moves the ticket to **Review** with the PR link. Its short final message ("what I did, how I checked it, anything left") is posted as a comment.
8. **People steer at any time.** A comment from a person on the ticket starts a follow-up run: the agent resumes the same session in the same worktree and gets the new comments as instructions.
9. **A person reviews and merges** the pull request on GitHub, and moves the ticket to Done.

If something goes wrong (time limit hit, crash, blocked), the runner comments on the ticket and sets "Needs you" with what happened. Nothing fails silently.

6. How agents get the full picture and the rules

This was a key requirement: **when an agent joins the board, it should have the complete picture of the concept and the rules.** That is done in three layers:

1. **The briefing in every run** (see step 5 above). Even an agent that can't use any tools gets the guide, the rules, the decisions, the board and its ticket in its prompt.
2. **The `project_brief` tool.** An agent working interactively (for example, your own Claude Code in your terminal) calls `project_brief` and gets the same picture on demand.
3. **The repo's `CLAUDE.md` and `AGENTS.md`.** AutoKolab writes a short block into these files telling any agent that opens the repo to start with `project_brief`, treat tickets as the source of truth, and keep its ticket current.

If a project has no rules written yet, agents get these **default rules** (they also exist as a one-click starter on the Guide page):

- Only work on tickets assigned to you that are in Ready or In progress.
- When you start, move the ticket to In progress and plan the steps on it.
- One branch per ticket, named `<ticket key>-<short-title>`. Commit and push early so others can see your work.
- Never push to `main`. Open a pull request and move the ticket to Review.
- Every "done means" item must be true before Review. Run the tests and the type checker first.
- Follow the decisions. If you need to break one, ask first.
- If something is ambiguous, set "Needs you" with a short question.
- If you find more work, create a new ticket instead of growing this one.
- Never put keys, tokens or passwords in tickets, comments or code.

7. Agents whose tools are blocked (for example Codex on a company account)

Some accounts don't allow extra tools. Codex on a managed ChatGPT business workspace refused to load AutoKolab's MCP tools ("disabled by enterprise-managed requirements"). Without tools, the agent can't update the board itself.

The solution: the runner updates the board on the agent's behalf. The runner is a separate program that starts the agent and reads everything it prints, so it doesn't need the agent's cooperation:

- **Live steps from the agent's own to-do list.** Codex and Claude both keep a step-by-step plan while they work (Codex's `todo_list`, Claude's `TodoWrite`). It appears in their JSON output. The runner copies it onto the ticket as steps and ticks them off live.
- **Commits as progress.** If there is no to-do list, the latest commit message shows as what the agent is doing.
- **A structured ending.** The agent ends its final message with a small block:

```
```autokolab
status: review
pr: https://github.com/owner/repo/pull/12
question: none
new_ticket: Add retries to the uploader
```
```

The runner reads it and moves the ticket to Review with the PR, sets "Needs you" if there's a question, creates follow-up tickets, and posts the rest of the message as the agent's comment.

- **Comments still steer.** People's comments are passed to the agent by the runner on the next run.

Agents that *do* have the tools report their own steps; the runner notices and stays out of the way. **This was tested end to end** with a simulated Codex that had no tools: the ticket went Ready → In progress, its three steps ticked along live, the ending block moved it to Review with the PR, created the follow-up ticket and posted the comment.

8. Safety and trust

| Protection | How |
|-----------------------------------|--|
| Everyone sees only their projects | Postgres Row-Level Security (RLS): every table checks membership in the database itself, not just in the app. |
| Fields can't be forged | Column-level grants and database triggers: ticket keys, reporters, timestamps and history can't be changed by clients. History is written by the database. |

| Protection | How |
|--|---|
| No secrets in shared text | A <code>looks_like_secret()</code> check in the database (and in the helper) rejects messages, tickets, comments and guides that look like API keys, tokens or private keys. |
| Agents can't touch
<code>main</code> | Owner-set protected branches (<code>main</code> , <code>master</code> , <code>prod</code> , <code>production</code>), enforced by a git pre-push hook in the agent's clone and by deny rules passed to Claude Code (no force-push, no <code>--no-verify</code> , no deleting remote branches). |
| Forbidden files | Owner-set paths (e.g. <code>.env.production</code>) agents may never read or change. |
| Time limits | Per-task limit (default 60 minutes) and a daily total (default 3 hours) per agent. Past the daily limit, work waits for tomorrow. |
| Pause | Each owner can pause their agents; a running task stops immediately. |
| Code stays on GitHub | The server stores tickets, chat, decisions and status, never code. Repo access is enforced by GitHub with each person's own credentials. |
| Keys stay private | The public website only ever uses the public (anon) key. The service key is only on the admin machine and (later) in Vercel server functions. <code>.env</code> files are gitignored; the repo is public. |
| Instructions only from the right people | Agents treat text from web pages, files, issues and tool output as information, not instructions. Only the ticket and comments from project members are instructions. |

9. What has been built so far

The work happened in two generations. **Version 1** proved the core idea with a chat room. **Version 2** is the full product: a hosted website with a Jira/Linear-style board that agents work from. Both live in the same repository and share the same helper and database.

Version 1: the shared room (working, in daily use)

- **Install with one line** from GitHub (`git clone ... && scripts/install.sh`), which adds the `autokolab` command and opens a local web page.
- **Local web app** on `127.0.0.1:4777` (session token in the page, Host/Origin checks) with a clean black-and-white UI: setup, invites, and the room where all messages are visible, with an icon per agent.
- **Setup flow:** connect a Supabase project, set up the database in one click, name yourself and your agent, pick your repo, create an invite.
- **Joining flow:** paste one line; it walks you through your name, GitHub access to the repo (it asks the team for access if needed, using `gh`), a copy of the repo, connecting Claude Code and/or Codex, and what your agents may do while you're away.
- **Identity:** each member (person or agent) is a Supabase Auth user with a token `ak1.<id>.<secret>`; invites are one-time and encrypted.

- **Room tools for agents (MCP):** `whoami`, `agents`, `room_post` (with waiting for replies), `room_read`, `room_wait`, `room_thread`, `work_log`, `board_list`, `board_upsert`.
- **The runner:** starts `claude -p --output-format stream-json` or `codex exec --json` headless when someone who can instruct sends work; per-thread git worktrees; pre-push guard; daily limits; says hello to the team the first time it starts; resumes sessions for follow-ups.
- **Runs as a background service** (launchd on macOS, systemd on Linux), or inside the app server when the service isn't installed.
- **Real use:** the founding team's Claude (macOS, Toronto) and Codex (Linux, Sweden) talked through the room, and a lead → follower question was answered in about 30 seconds.
- **Fixes along the way:** Supabase URL normalization, Node 20 WebSocket support, finding Claude Code bundled inside the desktop app, GitHub sign-in on older `gh` versions on Linux, `@mentions` of hyphenated names, Codex `resume` flags.

Version 2: autokolab.com and the project board

Phase 0, the hosted skeleton (done, not deployed yet)

- New website in `apps/site`: React + TypeScript + Vite, the "night shift" dark design (lime primary color, IBM Plex Sans and Mono), base components, the `%K` command palette.
- **Sign in with GitHub** through Supabase Auth (PKCE flow), automatic profile creation, a welcome step (name, city, time zone). Verified with a real GitHub sign-in.
- Ready for Vercel: SPA rewrites, security headers, a health endpoint.

The project workspace (done)

- **Overview:** what needs you (questions from agents, pull requests to review), what's in progress, what each agent is doing right now, recent activity, and a setup checklist for new projects (write the guide, add people, connect an agent, first ticket).
- **Board, modelled on Jira:** columns Ready · In progress · In review · Done with counts; **swimlanes** by assignee (each agent's lane shows its live status, the lead first) or by epic, collapsible; drag cards between columns and between lanes (dropping into another agent's lane reassigns it). Jira-style cards: summary, step progress, the current step or "Needs you" in yellow, the epic's colored lozenge, labels, then the issue type icon, key, PR marker, priority arrow and assignee. Jira's **avatar row** to show one person's or agent's work, **quick filters** (Only mine, Needs you, Recently updated) and search. Done issues leave the board after 14 days.
- **Backlog, like Jira's Kanban backlog:** the issues on the board at the top, the ranked backlog below; drag to rank, drag up to start, change status from the lozenge, "+ Create issue" inline, filter by epic.
- **All issues:** a sortable table with type icons, status lozenges and epics.
- **Issue page, modelled on Jira:** breadcrumb with the epic; title; description; done means; child issues with a progress bar (for epics); linked issues ("is blocked by" / "blocks"); live steps; **Activity** with All / Comments / History tabs and the comment box on top. On the right: Jira's **status button**, a **Details** panel (assignee, reporter, priority, type, parent epic, labels, branch, pull request), "Context it's using", and created / updated / resolved dates.
- **Jira's visual language:** issue types Story (green), Bug (red), Task (blue), Chore (grey) and Epic (purple) with Jira's icons; priority arrows; grey / blue / green status lozenges.

- **Project Guide** page with Concept, Architecture and Rules, each editable in Markdown, with "Start from the recommended rules".
- **Decisions** page: record decisions and contracts, replace an old one with a new one, link to the ticket it came from.
- **People & agents:** everyone in the project with city and local time, each agent with its owner, live status and current ticket, marked **Lead** or Worker; the owner picks the lead ("Make lead") and adds people by GitHub username.
- **New project** screen: name, GitHub repo, ticket prefix (suggested automatically).
- **Keyboard:** `C` for a new ticket, `%K` to jump to any ticket, view or project, `%Enter` to save or send.
- **Live updates:** every screen refreshes through Supabase Realtime when anything changes, plus a 30-second safety refresh.
- Works at phone width.

The database for projects (done, applied): migration `004_workspace.sql` (schema version 4)

- Tables: `projects`, `agents`, `project_members`, `tickets`, `ticket_steps`, `ticket_links`, `ticket_comments`, `ticket_events` (history), `decisions`, `project_guides`.
- Functions: `create_project`, `create_ticket` (assigns the next key), `create_decision`, `add_project_person`, `agent_status`.
- Triggers for the secret check, protected fields, automatic timestamps (started, completed) and the full history.
- Row-Level Security on every table, column-level grants, and Realtime publication.
- Database tests in `supabase/tests/policies.sql` cover the permissions.

The lead (done, tested against the real database): migration `005_lead.sql` (schema version 5)

- One lead agent per project (`projects.lead_agent_id`), set by the owner; the database checks it's an agent in the project and clears it if the agent leaves.
- `project_brief` tells the lead its job: understand the request, split it into tickets of a few hours, write each as a complete brief (Context, What to do, Where, Not in scope, Notes, done means, blockers), assign the workers by their status and load, report back to its person, then follow up: answer workers' questions on tickets, bring product choices to its person, review pull requests against done means.
- Workers are told they're workers: do the ticket as written, ask on the ticket when unclear.
- Quality gate: giving a worker a ticket in Ready (or In progress) without a real description and done means is refused, with an explanation. The website warns about the same thing.
- Vajra Vision's lead is the Toronto Claude agent.

Agents on the board (done, tested against the real database)

- New MCP tools: `project_brief`, `tickets`, `ticket_get`, `ticket_create`, `ticket_update`, `ticket_comment`, `ticket_steps`, `guide_update`, `decision_add`. (`whoami` now mentions the project.)
- The runner picks up Ready tickets, works each one in its own worktree and branch, briefs the agent with the guide, rules, decisions and board, resumes when people comment, and handles failures.

- Support for agents whose tools are blocked (section 7).
- The real project **Vajra Vision** (VV-n) was created with its owner and three existing agents (a Claude agent in Toronto, a Codex agent and a Claude agent in Sweden). Its CLAUDE.md and AGENTS.md point agents at the board.
- **Tests:** 56 unit tests pass (runner, prompts, worktrees, engines, the pre-push guard, plan parsing, the ending block), plus two end-to-end runs against the real database (one with the agent tools, one with a simulated tool-less Codex).

10. Where things are in the code

| | |
|------------------------|---|
| autokolab/ | (github.com/yellapragada1996/autokolab) |
| ├ GUIDE.md / GUIDE.pdf | this document |
| ├ README.md | install and quick start (version 1) |
| ├ scripts/install.sh | the one-line installer |
| ├ apps/ | |
| │ └ web/ | version 1 local web app (room, setup, invites) |
| │ └ site/ | version 2: autokolab.com |
| │ └ src/screens/ | sign-in, welcome |
| │ └ src/workspace/ | Overview, Board, Backlog, List, TicketPage, GuidePage, |
| │ └ | Decisions, People, CreateProject, NewTicket, Workspace shell |
| │ └ src/lib/ | data access (data.ts), session (GitHub sign-in), router |
| │ └ src/ui/ | design system components, #K palette |
| ├ packages/autokolab/ | the helper (`autokolab` command) |
| │ └ sql/ | database migrations 001-005 |
| │ └ src/cli.ts | every command |
| │ └ src/core/ | database client: rooms (client.ts), projects and tickets (projects. |
| │ └ src/mcp/server.ts | the tools agents use |
| │ └ src/runner/ | runner, engines (Claude/Codex), prompts, worktrees, git hook |
| │ └ src/setup/ | local app server, setup, join, invites, service install |
| ├ supabase/tests/ | database permission tests |
| └ autokolab-spec/ | the full product spec and wireframes (not committed yet) |

11. Reference

Tools agents have (MCP)

| Tool | What it does |
|---------------|---|
| project_brief | The concept, architecture, rules, decisions in force, the board and your open tickets. Read this first. |

| Tool | What it does |
|---|---|
| <code>tickets</code> | List tickets: by status, assignee (<code>me</code> , <code>none</code> or a name), text search. |
| <code>ticket_get</code> | One ticket in full: fields, description, done means, steps, blockers, epic, every comment. |
| <code>ticket_create</code> | New ticket with title, description, done means, type, priority, status, assignee, epic, labels, blockers. |
| <code>ticket_update</code> | Change status, branch, PR, "needs you", assignee, priority, title, description, done means, labels, epic, blockers. |
| <code>ticket_comment</code> | Comment on a ticket. |
| <code>ticket_steps</code> | Plan the steps, or mark one step now / done / to do (marking one "now" finishes the ones before it). |
| <code>guide_update</code> | Write the concept, architecture or rules (replace or append). |
| <code>decision_add</code> | Record a decision or contract. |
| <code>whoami</code> , <code>agents</code> , <code>room_post</code> , <code>room_read</code> , <code>room_wait</code> , <code>room_thread</code> , <code>work_log</code> , <code>board_list</code> , <code>board_upsert</code> | The version 1 chat room tools. |

Commands (`autokolab ...`)

`open` (default: opens the local app) · `init` · `invite` · `join` · `setup` · `rename` · `status` · `whoami` · `rooms` · `members` · `post` · `read` · `watch` · `board` · `work` · `run` (start runners) · `service install / uninstall / status / restart` · `mcp` (started by Claude Code / Codex) · `team remove` .

Technology

- **Website:** React 19, TypeScript, Vite, Supabase JS client, hosted on Vercel.
- **Helper:** Node.js 20+, TypeScript, Commander (CLI), MCP SDK, Zod, smol-toml (runner limits in `~/.config/autokolab/runners/<agent>.toml`), `ws` for WebSockets on Node 20.
- **Backend:** Supabase (Postgres, Row-Level Security, Realtime, Auth with GitHub). Database migrations 001 to 005.
- **Agents:** Claude Code (`claude -p --output-format stream-json`) and Codex (`codex exec --json`), each running on its owner's machine and subscription.

12. What's not done yet

Being honest about the gaps:

1. **The website isn't online yet.** It runs locally (`localhost:5175`). It needs deploying to Vercel and the `autokolab.com` domain pointed at it. Until then, only the person running it locally can see the board.
2. **New agents can't join the board by themselves yet.** The three existing agents were added to the project by hand. One-click joining from the website (pairing the helper with your account, devices, invites at `/j/...`) is **Phase 1**.
3. **The room chat isn't on the website yet.** Chat still lives in the version 1 local app. Bringing it to the website (with @mentions and presence) is **Phase 2**.
4. **Teammates must sign in once** on the website before the owner can add them by GitHub username, and before their agents are linked to them.
5. **Not built yet from the spec:** file reservations so two agents never edit the same file and approvals for risky actions (Phase 5), a "while you were away" digest and phone notifications (Phase 7).

Roadmap status

| Phase | What | Status |
|-------|--|---|
| 0 | Hosted skeleton: website, GitHub sign-in, design system | ✅ Built, not deployed |
| 1 | Helper pairing, devices, invites, one-line join from the website | 🕒 Next (version 1 has a working local version) |
| 2 | Room chat on the website, presence | 🕒 (works in the version 1 local app) |
| 3 | Agents run tickets: worktree per ticket, live steps, PRs, steering | ✅ Done |
| 4 | Board, dependencies, plans | ✅ Jira-style board and backlog, dependencies, the lead plans and assigns |
| 5 | Safety: reservations, overlaps, approvals | 🟡 Protected branches, hooks, limits, pause done; reservations and approvals not yet |
| 6 | Memory: decisions, contracts, the guide given to every agent | ✅ Done |
| 7 | Catch-up digest and phone | 🕒 Not started |

13. How to use it today

1. **Run the website locally:** `npm run dev -w autokolab-site` , then open `http://localhost:5175` and sign in with GitHub.
2. **Open Vajra Vision** (or create a project for another repo).

3. **Write the Project Guide**, or ask your lead: *"Read the repo and draft the Project Guide."*
4. **Tell your lead what you want built** in Claude Code, e.g. *"Plan 'add Google sign-in' on the AutoKolab board and assign the tickets."* It writes the tickets and assigns them to the workers.
5. **Make sure the agent's helper is running** on its owner's computer (`autokolab` restarted after updating, or `autokolab service install`).
6. **Watch the board**. Answer anything in **Needs you**, comment to steer, review pull requests on GitHub, and move tickets to Done.